JQuery

jQuery

What is jQuery ?

1. jQuery is a JavaScript Library.
2. Write Less do more
3. jQuery greatly simplifies JavaScript programming.
4. The purpose of jQuery is to make it much easier to use JavaScript on your website.
5. jQuery takes a lot of common tasks that require many lines of JavaScript code to accomplish, and wraps them into methods that you can call with a single line of code.

What is jQuery?

☞ jQuery is a lightweight JavaScript library for easily access dom elements and add action to it.

Write less, Do more

☞ In simple words, jQuery is the shortcut way to write JavaScript code.

jQuery

1. Purpose of jQuery -

(a) Simplify HTML DOM tree traversal and manipulation, as well as event handling, CSS animation, and Ajax.

(b) jQuery will run exactly the same in all major browsers. (browser compatibility)

JavaScript Code

```javascript
const element = document.getElementById("btn");

element.addEventListener("click", function () {
    document.getElementById("output").innerHTML = "Good Morning!";
});
```

jQuery Code

```javascript
$("#btn").click(function () {
    $("#output").html("Good Morning!");
});
```

jQuery features

- DOM manipulation
- Event listners
- Animations
- Much more
- Style manipulation
- Effects
- Ajax (HTTP requests)

Adding jQuery to Your Web Pages

There are several ways to start using jQuery on your web site:

- Download the jQuery library from jQuery.com
- Include jQuery from a CDN, like Google etc

jQuery Syntax

The jQuery syntax is tailor-made for **selecting** HTML elements and performing some **action** on the element(s).

Basic syntax is: **$(*selector*).*action*()**

- A $ sign to define/access jQuery
- A (*selector*) to "query (or find)" HTML elements
- A jQuery *action*() to be performed on the element(s)

The Document Ready Event

```
$(document).ready(function(){

  // jQuery methods go here...

});
```

This is to prevent any jQuery code from running before the document is finished loading (is ready).

The jQuery team has also created an even shorter method for the document ready event:

```
$(function(){

  // jQuery methods go here...

});
```

jQuery Selectors

Syntax	Description
$("*")	Selects all elements
$(this)	Selects the current HTML element
$("p.intro")	Selects all <p> elements with class="intro"
$("p:first")	Selects the first <p> element
$("ul li:first")	Selects the first element of the first
$("ul li:first-child")	Selects the first element of every
$("[href]")	Selects all elements with an href attribute
$("a[target='_blank']")	Selects all <a> elements with a target attribute value equal to "_blank"
$("a[target!='_blank']")	Selects all <a> elements with a target attribute value NOT equal to "_blank"
$(":button")	Selects all <button> elements and <input> elements of type="button"
$("tr:even")	Selects all even <tr> elements
$("tr:odd")	Selects all odd <tr> elements

jQuery Event Methods

An event represents the precise moment when something happens.
Examples:
•moving a mouse over an element
•selecting a radio button
•clicking on an element
•Pressing a key on keyboard

Here are some common DOM events:

Mouse Events	Keyboard Events	Form Events	Document/Window Events
click	keypress	submit	load
dblclick	keydown	change	resize
mouseenter	keyup	focus	scroll
mouseleave		blur	unload

jQuery Syntax For Event Methods

```
$("p").click(function(){
  // action goes here!!
});
```

Example

```
$("p").click(function(){
  $(this).hide();
});
```

Example

```
$("#p1").mouseleave(function(){
  alert("Bye! You now leave p1!");
});
```

hover()

The `hover()` method takes two functions and is a combination of
the `mouseenter()` and `mouseleave()` methods.
The first function is executed when the mouse enters the HTML element, and the second function is executed when the mouse leaves the HTML element:

Example

```
$("#p1").hover(function(){
  alert("You entered p1!");
},
function(){
  alert("Bye! You now leave p1!");
});
```

Example

```
$("input").focus(function(){
  $(this).css("background-color", "#cccccc");
});
```

The on() Method

The `on()` method attaches one or more event handlers for the selected elements.
Attach a click event to a `<p>` element:

Example

```
$("p").on({
  mouseenter: function(){
    $(this).css("background-color", "lightgray");
  },
  mouseleave: function(){
    $(this).css("background-color", "lightblue");
  },
  click: function(){
    $(this).css("background-color", "yellow");
  }
});
```

jQuery Effects

- Hide and Show
- Fade
- Slide
- Animate
- Chaining

```
$(selector).hide(speed,callback);

$(selector).show(speed,callback);

$(selector).toggle(speed,callback);
```

The optional speed parameter can take the following values: "slow", "fast", or milliseconds.

The optional callback parameter is a function to be executed after the `hide()` or `show()` method completes

jQuery has the following fade methods:
- `fadeIn()`
- `fadeOut()`
- `fadeToggle()`
- `fadeTo()`

Syntax:

```
$(selector).fadeIn(speed,callback);

$(selector).fadeOut(speed,callback);

$(selector).fadeToggle(speed,callback);

$(selector).fadeTo(speed,opacity,callback);
```

```
$("button").click(function(){
  $("#div1").fadeToggle();
  $("#div2").fadeToggle("slow");
  $("#div3").fadeToggle(3000);
});
```

jQuery Sliding Methods

With jQuery you can create a sliding effect on elements.
jQuery has the following slide methods:

- slideDown()
- slideUp()
- slideToggle()

Syntax:

```
$(selector).slideUp(speed,callback);

$(selector).slideDown(speed,callback);

$(selector).slideToggle(speed,callback);
```

NOTE: 1. The optional speed parameter can take the following values: "slow", "fast", milliseconds.
2. The optional callback parameter is a function to be executed after the sliding completes.

Example

```
$("#flip").click(function(){
  $("#panel").slideUp();
});
```

Example

```
$("#flip").click(function(){
  $("#panel").slideDown();
});
```

Example

```
$("#flip").click(function(){
  $("#panel").slideToggle();
});
```

jQuery Effects - Animation

The jQuery `animate()` method is used to create custom animations.

Syntax:

```
$(selector).animate({params},speed,callback);
```

- The required params parameter defines the CSS properties to be animated.
- The optional speed parameter specifies the duration of the effect. It can take the following values: "slow", "fast", or milliseconds.
- The optional callback parameter is a function to be executed after the animation completes.

Example

```
$("button").click(function(){
  $("div").animate({left: '250px'});
});
```

NOTE: By default, all HTML elements have a static position, and cannot be moved.
To manipulate the position, remember to first set the CSS position property of the element to relative, fixed, or absolute!

Example 2

```
$("button").click(function(){
  var div = $("div");
  div.animate({left: '100px'}, "slow");
  div.animate({fontSize: '3em'}, "slow"
);
});
```

Example 1

```
$("button").click(function(){
  $("div").animate({
    left: '250px',
    opacity: '0.5',
    height: '150px',
    width: '150px'
  });
});
```

jQuery Callback Functions

A callback function is executed after the current effect is 100% finished.
JavaScript statements are executed line by line. However, with effects, the next line of code can be run even though the effect is not finished. This can create errors.

To prevent this, callback function can be created.

A callback function is executed after the current effect is finished.

Typical syntax: **$(*selector*).hide(*speed,callback*);**

Example with Callback

```
$("button").click(function(){
  $("p").hide("slow", function(){
    alert("The paragraph is now hidden");
  });
});
```

Example without Callback

```
$("button").click(function(){
  $("p").hide(1000);
  alert("The paragraph is now hidden");
});
```

jQuery - Chaining

Chaining allows us to run multiple jQuery methods (on the same element) within a single statement.

Example

```
$("#p1").css("color", "red").slideUp(2000).slideDown(2000);
```

jQuery DOM Manipulation

Three simple, but useful, jQuery methods for DOM manipulation are:

- `text()` - Sets or returns the text content of selected elements
- `html()` - Sets or returns the content of selected elements (including HTML markup)
- `val()` - Sets or returns the value of form fields

jQuery - Get Content and Attributes

Example

```
$("#btn1").click(function(){
  alert("Text: " + $("#test").text());
});
$("#btn2").click(function(){
  alert("HTML: " + $("#test").html());
});
```

Example

```
$("#btn1").click(function(){
  alert("Value: " + $("#test").val());
});
```

Example

```
$("button").click(function(){
  alert($("#anchor").attr("href"));
});
```

jQuery - Set Content and Attributes

We will use the same three methods from the to **set content**:

- `text()` - Sets or returns the text content of selected elements
- `html()` - Sets or returns the content of selected elements (including HTML markup)
- `val()` - Sets or returns the value of form fields

Example

```
$("#btn1").click(function(){
  $("#test1").text("Hello world!");
});
$("#btn2").click(function(){
  $("#test2").html("<b>Hello world!</b>");
});
$("#btn3").click(function(){
  $("#test3").val("Dolly Duck");
});
```

jQuery Manipulating CSS

jQuery has several methods for CSS manipulation. We will look at the following methods:

- `addClass()` - Adds one or more classes to the selected elements
- `removeClass()` - Removes one or more classes from the selected elements
- `toggleClass()` - Toggles between adding/removing classes from the selected elements
- `css()` - Sets or returns the style attribute

Example

```
$("button").click(function(){
  $("h1, h2, p").addClass("blue");
  $("div").addClass("important");
});
```

Example

```
$("button").click(function(){
  $("h1, h2, p").toggleClass("blue");
});
```

Example

```
$("button").click(function(){
  $("h1, h2,
p").removeClass("blue");
});
```

jQuery css() Method

The `css()` method sets or returns one or more style properties for the selected elements.

Return a CSS Property

To return the value of a specified CSS property, use the following syntax:

```
css("propertyname");
```

Example

```
$("p").css("background-color");
```

Set a CSS Property

To set a specified CSS property, use the following syntax:

```
css("propertyname","value");
```

Example

```
$("p").css("background-color", "yellow");
```

Set Multiple CSS Properties

To set multiple CSS properties, use the following syntax:

```
css({"propertyname":"value","propertyname":"value",...});
```

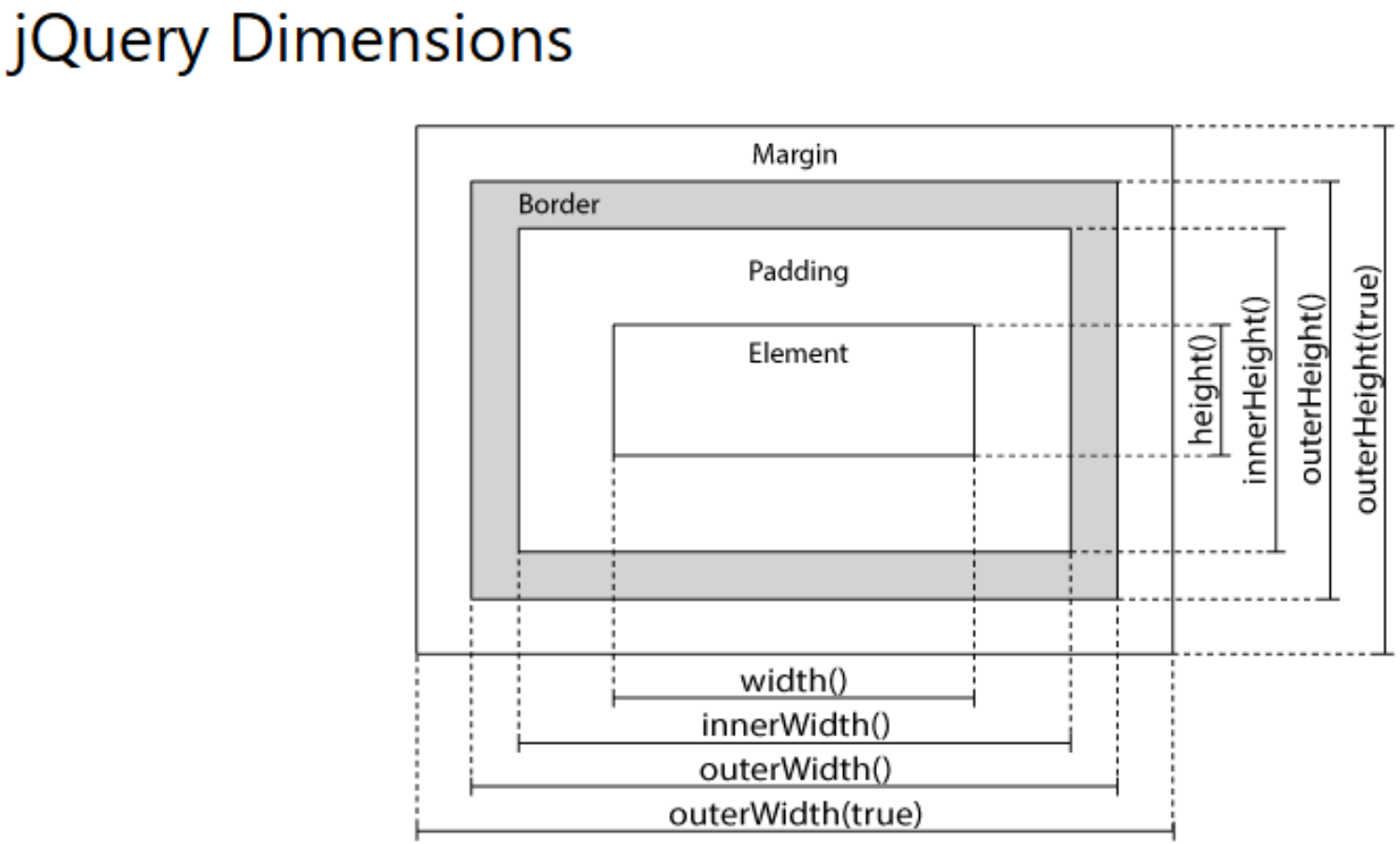
Example

```
$("p").css({"background-color": "yellow", "font-size": "200%"});
```

jQuery Dimension Methods

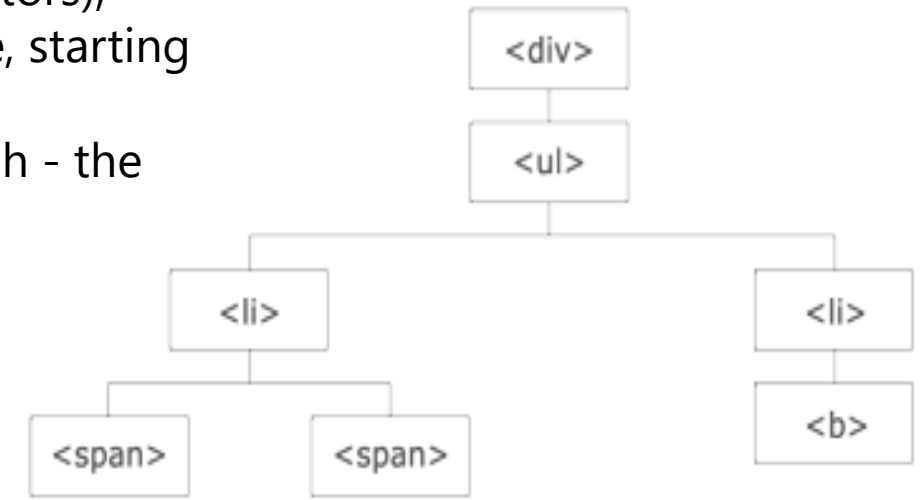
jQuery has several important methods for working with dimensions:

- `width()`
- `height()`
- `innerWidth()`
- `innerHeight()`
- `outerWidth()`
- `outerHeight()`



jQuery Traversing

- jQuery traversing, which means "move through", are used to "**find" (or select) HTML elements based on their relation to other elements**.
- Start with one selection and move through that selection until you reach the elements you desire.
- With jQuery traversing, one can easily move up (ancestors), down (descendants) and sideways (siblings) in the tree, starting from the selected (current) element.
- This movement is called traversing - or moving through - the DOM tree.



Traversing Up the DOM Tree

Three useful jQuery methods for traversing up the DOM tree are:

- `parent()`
- `parents()`
- `parentsUntil()`

jQuery parent() Method

The `parent()` method returns the direct parent element of the selected element.
This method only traverse a single level up the DOM tree.

Example

```
$(document).ready(function(){
  $("span").parent();
});
```

```
<script>
$(document).ready(function(){
 $("span").parent().css({"color": "red", "border": "2px solid red"});
});
</script>
```

jQuery parents() Method

The `parents()` method returns all ancestor elements of the selected element, all the way up to the document's root element (`<html>`).

Example

```
$(document).ready(function(){
  $("span").parents();
});
```

jQuery parentsUntil() Method

The `parentsUntil()` method returns all ancestor elements between two given arguments.

Example

```
$(document).ready(function(){
  $("span").parentsUntil("div");
});
```

jQuery Traversing - Descendants

A descendant is a child, grandchild, great-grandchild, and so on.
Traversing Down the DOM Tree

Two useful jQuery methods for traversing down the DOM tree are:
`children()` `find()`

The `children()` method returns all direct children of the selected element.

Example

```
$(document).ready(function(){
  $("div").children();
});
```

The `find()` method returns descendant elements of the selected element, all the way down to the last descendant.

The following example returns all elements that are descendants of :

```
$(document).ready(function(){
  $("div").find("span");
});
```

jQuery Traversing - Siblings

Traversing Sideways in The DOM Tree

There are many useful jQuery methods for traversing sideways in the DOM tree:

- `siblings()`: The `siblings()` method returns all sibling elements of the selected element
- `next()`: The next() method returns the next sibling element of the selected element.
- `nextAll()`: The nextAll() method returns all next sibling elements of the selected element.
- `nextUntil()`: The nextUntil() method returns all next sibling elements between two given arguments.
- `prev()`
- `prevAll()`
- `prevUntil()`

The `prev()`, `prevAll()` and `prevUntil()` methods work just like the methods above but with reverse functionality

```
<body class="siblings">
<div>div (parent)
 <p>p</p>
 <span>span</span>
 <h2>h2</h2>
 <h3>h3</h3>
 <h4>h4</h4>
 <h5>h5</h5>
 <h6>h6</h6>
 <p>p</p>
</div>
</body>
```

jQuery Traversing - Filtering

The first(), last(), eq(), filter() and not() Methods

The `first()` method returns the first element of the specified elements.
The following example selects the first `<div>` element:

```
$(document).ready(function(){
  $("div").first();
});
```

The `last()` method returns the last element of the specified elements.
The following example selects the last `<div>` element:

```
$(document).ready(function(){
  $("div").last();
});
```

The `eq()` method returns an element with a specific index number of the selected elements.
The index numbers start at 0, so the first element will have the index number 0 and not 1. The following example selects the second `<p>` element (index number 1):

```
$(document).ready(function(){
  $("p").eq(1);
});
```

The `filter()` method lets you specify a criteria. Elements that do not match the criteria are removed from the selection, and those that match will be returned.
The following example returns all `<p>` elements with class name "intro":

```
$(document).ready(function(){
  $("p").filter(".intro");
});
```

The `not()` method returns all elements that do not match the criteria.
The following example returns all `<p>` elements that do not have class name "intro":

```
$(document).ready(function(){
  $("p").not(".intro");
});
```

jQuery - The noConflict() Method

What if other JavaScript frameworks also use the $ sign as a shortcut?
If two different frameworks are using the same shortcut, one of them might stop working.
The jQuery team have already thought about this, and implemented the `noConflict()` method.

The `noConflict()` method releases the hold on the $ shortcut identifier, so that other scripts can use it.
You can of course still use jQuery, simply by writing the full name instead of the shortcut:

Example

```
$.noConflict();
jQuery(document).ready(function(){
  jQuery("button").click(function(){
    jQuery("p").text("jQuery is still working!");
  });
});
```

You can also create your own shortcut very easily.
The `noConflict()` method returns a reference to jQuery, that you can save in a variable, for later use.
Here is an example:

Example

```
var jq = $.noConflict();
jq(document).ready(function(){
  jq("button").click(function(){
    jq("p").text("jQuery is still working!");
  });
});
```